

1. Trustworthy Person Algorithm:
 1. Initialize integer variables i to 0, j to 1, and n to $T.length$
 2. While i and j are less than or equal to n
 - a. if($T[i][j] == 1$) then increment i .
 - b. else if($T[i][j] == 0$) then increment j .
 - c. Else there is an error in the matrix, though not necessary as matrix should be all 1s and 0s.
 3. Check if the i value is greater than n , and if it is, return -1.
 4. Create a for loop that iterates from $x = 1$ to n , which does...
 - a. Checks if $T[i][x] != 0$, and if it does, it returns -1.
 5. Create another for loop that iterates from $y = 1$ to n , which does...
 - a. Checks if $k != i$ and $T[y][i] != 1$, and if it does, return -1.
 6. Finally, return the index ' i ' if all of the falsifiable checks have passed successfully.

Total Runtime = $O(n)$

- (1) Initializing the integer variables should take $O(1)$ time, as it is a constant operation.
- (2) This while loop has a time of $O(n)$ and not $O(n^2)$ because i and j are both incremented in the for loop. This means that the maximum number of iterations that are possible in the worst case is $2n$, because i would be incremented n times and j would be incremented n times.
- (3) This if check will be performed in $O(n)$ time, as it simply accesses the i value and checks if it is smaller than n .
- (4-5) Both of these for loops would equate to a runtime of $O(n)$ each, because there are a total of $n-1$ iterations and n executions of each for loop. There are also no inner loops in either loop, and no further access of memory outside of accessing the i value..
- (6) Returning takes $O(1)$ time.

2. Treasure Hunt Algorithm:

- Create a hash table that keeps track of the degree of each island, or rather the amount of bridges connected to each island. Also create an empty hashtable hashIsland that will keep track of surrounding islands later, preferably with a competent hash function and a starting size of the amount of islands.
 - **Run-Time = $O(1)$** → initialization is constant operation.
- Iterate through the list of bridges (B) using a for loop and increment the degree amount for the islands that are interlinked by each bridge if the island belongs to the pair of islands that belong to the bridge.
 - **Run-Time = $O(|B|)$** → Iterated through $|B|$ items representing the amount of bridges/edges.
- Iterate through the hash table using a for loop and find the highest degreed island, or the island with the most bridges connected to it.
 - **Run-Time = $O(|I|)$** → Hash table size = $|I|$ and that is how many items that were iterated through, which also represents the amount of total islands there are.
- Perform the first round of BFS to identify the islands that are connected to the initial island. The islands with a depth of one will be the ones connected to it. When these are found in the first round of BFS, add them to hashIsland that was initialized. This way, the neighboring islands are kept track of and can be returned later.
 - **Run-Time = $O(|I| + |B|)$** → Iterating through the graph by BFS results in a run time equal to the amount of islands combined with the amount of bridges.
- Return the initial island and hashIsland.
 - **Run-Time = $O(1)$** → Returning the values is a constant operation.

$$\text{Total Runtime} = O(1) + O(|B|) + O(|I|) + O(|B| + |I|) + O(1) = O(|I| + |B|)$$

3. Scheduling Courses Algorithm:

1. Initialize a directed graph G that entails that each node is a course and each directed edge from one course to another indicates that a course is a prerequisite for another course.
2. For each node in G , calculate the in-degree that represents the amount of edges/prerequisites that a course/node has.
3. Initialize an empty list `courseList` that will eventually store the courses in a proper order.
4. Initialize a queue Q that stores courses with no prerequisites and an in-degree = 0.
5. While the Q is not empty {
 - Dequeue a course, say c , from Q and add c to `courseList`.
 - For each course, say b , that c is a prerequisite for (neighbor of c in G) {
 - Decrement the in-degree of b by 1.
 - If the degree = 0, enqueue b into Q . }}
6. If size of `courseList` isn't equal to size of C , then a cycle is present and return false.
7. Initialize an empty hashmap `courseMap` to track the earliest semester a course can be taken.
8. For each course c in `courseList`
 - Set `courseMap[c]` to highest semester value of any prerequisites plus one. If a course in `courseList` does not have any prerequisites, then set their semester value to 1.
9. Initialize int variable `currentSemester` to 1 and iterate through `courseList`, stopping at last element. Increment `currentSemester` when a set of courses are completed within same semester.
10. For each set s in S
 - Set integer variable `lastSemester` to the latest semester possible among all prerequisites of courses in s using the hashmap.
 - If any course in s has a prerequisite not in the hashmap or its `courseMap[c]` value is greater than `lastSemester`, return False.
11. For each set s in S
 - If any two courses in the set have different `courseMap[c]` values, return false.
12. Return true.

Total Runtime = $O(V + E + S * T)$ —IDK if T is needed, but putting it down.

- V can be represented as C , E can be represented as **Pre**
- (1-2) Initializing the graph = $O(1)$, distinguishing the in-degree of the nodes takes $O(|V| + |E|)$ because each edge of each node must be checked.
- (3-4) Initializing the list and queue will take $O(|V|)$ because every course must be checked.

- (5) While loop and inside for loop visits each edge and node exactly once, therefore it takes $O(|V| + |E|)$.
- (6-8) Checking courseList size = $O(1)$, initializing and setting courseMap takes $O(V)$ due to iterating through courseList.
- (9) $O(V)$ time due to all courses being iterated through.
- (10) $O(S * T)$ time. Every set in s is iterated through, while **T represents the most amount of classes that can be simultaneous.**
- (11-12) Checking if courses in a set have different courseMap values will result in $O(S * T)$ time also. Returning true takes $O(1)$ time.

4. Minimum Semester Scheduling Algorithm:

1. Initialize a directed graph G that creates a node for each course c in C and creates a directed edge from c_1 to c_2 for each prerequisite connection between course c_1 and c_2 .
2. Initialize an empty list `courseOrder` that will eventually store courses in proper order.
3. Use DFS to topologically sort graph G and store the ordered results into `courseOrder`. If a cycle is detected, then return -1 to signify that the courses in C are impossible to take.
4. Initialize array `earliestSemester` of size $|C|$. Array measures the earliest semester a course can be taken through prerequisite structure. All values initialized to 1.
5. For each course c in `courseOrder`
 - For each course b that c is a prerequisite for through directed edges in G . Set the `earliestSemester[b]` to the maximum of `earliestSemester[b]`, `earliestSemester[b] + 1`.
6. For each set s in $S...$
 - Initialize an integer `maxSemesterCourse` and assign it to the course with the highest `earliestSemester[]` in set s .
 - For each course c in set $s...$
 - If `earliestSemester[c] < earliestSemester[maxSemesterCourse]`, set `earliestSemester[c]` to `earliestSemester[maxSemesterCourse]`.
 - For each course b that c is a prerequisite for.
 - Update `earliestSemester[d]` since c 's may have increased.
7. Find the maximum value in `earliestSemester[]` which portrays the minimum semester amount required to complete the classes. This requires iterating through the array.
8. Return the maximum value found in `earliestSemester[]`.

Total Runtime = $O(V + E + S * T)$ —IDK if T is needed, but putting it down.

- V can be represented as **C**, E can be represented as **Pre**
- (1-2) Creating the graph takes $O(V + E)$ time, initializing the list takes $O(1)$ time.
- (3) Topological sorting takes $O(V + E)$ time as each node and edge are checked/processed once.
- (4) Initializing earliestSemester array takes $O(V)$ time due to each vertice/course being visited once and set to 1.
- (5) Computing the earliest semester values will take $O(V + E)$ time because all vertices (courses) and edges (prerequisite connections) will be accessed in order to keep incrementing necessary semester values.
- (6) Takes $O(S * T)$ time, where T represents the maximum number of classes that can be in a simultaneous set. This is due to the loop accessing every course in every set. The prerequisite update for loop does not affect further memory because the courses are already topologically sorted and each course is only updated at most ONCE. So it does not become quadratic.
- (7-8) Iterating through the array will take $O(V)$ time due to there being V elements in the array that must be searched. Returning the number will equate to $O(1)$ time.